==========
1. ANALYTICAL PROBLEM SOLVING
==========

Approach:
Create the engineering dataset as a data frame, inspect it using summary() and str(), then apply the required data frame operations such as reshaping, merging or filtering.

R Code:

```
df <- data.frame(
ID=1:5,
Temperature=c(35,38,40,36,39),
Pressure=c(100,110,105,98,102)
)

str(df)
summary(df)
```

Justification:
The solution uses built-in R data frame functions which are efficient, easy to maintain, and suitable for engineering data analysis. Data frames allow easy filtering, summarization, reshaping and merging while preserving data integrity.

==========
2. MAKING DATA FRAMES
==========

Answer:

```
sensor_id <- c("S1","S2","S3","S4")
voltage <- c(3.1,3.4,3.5)
status <- c(TRUE,FALSE,TRUE,TRUE)

lengths <- c(length(sensor_id),
        length(voltage),
        length(status))

print(lengths)

max_len <- max(lengths)

padNA <- function(x,n){
if(length(x)<n){
```

```
missing <- (length(x)+1):n
cat("Missing Index:",missing,"\n")
x <- c(x,rep(NA,n-length(x)))
}
return(x)
}

sensor_id <- padNA(sensor_id,max_len)
voltage <- padNA(voltage,max_len)
status <- padNA(status,max_len)

df <- data.frame(sensor_id,
                 voltage,
                 status)

print(df)
```

Justification:

The routine first detects unequal vector lengths, reports missing positions and pads only the shorter vectors with NA before constructing the data frame. This prevents recycling errors and preserves data consistency.

==========
3. WORKING WITH DATA FRAMES
==========

Answer:

```
df <- data.frame(
Temperature=c("25","30","ERR_99","28","32","ERR_99")
)

invalid <- grep("[A-Za-z]",df$Temperature)

print(invalid)

table(df$Temperature[invalid])

df$Temperature[df$Temperature=="ERR_99"] <- NA

df$Temperature <- as.numeric(df$Temperature)

mean(df$Temperature,na.rm=TRUE)
```

Justification:

Character values prevent numeric calculations because the entire column becomes character type. Replacing invalid strings with NA and converting back to numeric restores correct statistical analysis.

==========
4. DATA RESHAPING
==========

Answer:

```
library(reshape2)

df <- data.frame(
Time=c(1,2,3),
SensorA_X=c(10,11,12),
SensorA_Y=c(5,6,7),
SensorB_X=c(20,21,22),
SensorB_Y=c(8,9,10)
)

long_df <- melt(df,
id.vars="Time",
variable.name="Sensor",
value.name="Value")

print(long_df)
```

Justification:

Long format stores each measurement as one observation, making regression, visualization and time-series analysis easier while preserving row relationships.

==========
5. MELTING AND CASTING OF DATA
==========

Answer:

```
library(reshape2)

molten <- melt(df,
```

```
id.vars=c("Timestamp","Sensor"))

mean_df <- dcast(
molten,
Timestamp~Sensor,
value.var="value",
fun.aggregate=mean
)

var_df <- dcast(
molten,
Timestamp~Sensor,
value.var="value",
fun.aggregate=var
)

print(mean_df)
print(var_df)
```

Justification:

Duplicate ID combinations create ambiguity because multiple values map to one output cell. Using fun.aggregate resolves conflicts by summarizing duplicate observations. Computing both mean and variance preserves central tendency as well as variability.

==========
6. MERGING DATA FRAMES
==========

Answer:

```
df_control$Timestamp <- round(df_control$Timestamp)
df_telemetry$Timestamp <- round(df_telemetry$Timestamp)

merged_df <- merge(
df_control,
df_telemetry,
by=c("SystemID","Timestamp"),
all=TRUE,
suffixes=c("_Control","_Telemetry")
)

print(merged_df)
```

Justification:

Exact timestamp matching causes many records to be lost due to clock drift. Rounding timestamps before merging minimizes data loss, while an outer join (all=TRUE) preserves unmatched records for later analysis. Renaming conflicting Status columns avoids ambiguity.

==========
DATA FRAMES - Q1
==========

Question:
Analyze a dataset of 50,000 student records stored in a data frame. Identify the most efficient method to detect duplicate entries and justify your approach.

Answer:

```
dup_rows <- duplicated(students)

students[dup_rows, ]
```

Justification:

The duplicated() function efficiently detects duplicate records with linear time complexity. It is faster and more memory-efficient than comparing every row manually, making it suitable for large datasets.

==========
DATA FRAMES - Q2
==========

Question:
A data frame contains missing values across multiple columns. Evaluate different strategies for handling missing data and recommend the best solution.

Answer:

```
is.na(df)

na.omit(df)

df$Marks[is.na(df$Marks)] <-
median(df$Marks,na.rm=TRUE)
```

Justification:

Removing missing records is suitable when only a few values are missing. Median imputation is preferred for skewed data because it is less affected by outliers. The best strategy depends on the percentage and pattern of missing data.

==========
DATA FRAMES - Q3
==========

Question:
Given a sales data frame with inconsistent data types, analyze the impact on statistical calculations and propose corrective actions.

Answer:

```
str(sales)

sales$Amount <-
as.numeric(sales$Amount)

sales$Quantity <-
as.integer(sales$Quantity)
```

Justification:

Incorrect data types prevent mathematical calculations and may produce incorrect results. Converting each column to the correct data type ensures accurate statistical analysis and improves processing efficiency.

==========
DATA FRAMES - Q4
==========

Question:
Compare two large data frames containing customer transactions. Determine how to identify data integrity issues and evaluate their causes.

Answer:

```
compare <- merge(
df1,
df2,
by="CustomerID",
all=TRUE
```

)

summary(compare)

duplicated(compare)

Justification:

Comparing merged datasets helps identify duplicate records, missing transactions and inconsistent values. Integrity issues may arise from duplicate keys, missing records, incorrect data entry or synchronization problems.

==========
DATA FRAMES - Q5
==========

Question:
A company reports conflicting results from the same data frame. Analyze possible reasons and design a validation framework.

Answer:

str(df)

summary(df)

any(is.na(df))

duplicated(df)

stopifnot(nrow(df)>0)

Justification:

Conflicting results may occur due to missing values, duplicate records, incorrect data types or inconsistent filtering. A validation framework should include structure verification, missing-value detection, duplicate checking and summary statistics before performing analysis.

==========
MAKING DATA FRAMES - Q1
==========

Question:

Design a data frame structure for a smart-city traffic monitoring system and justify the selection of attributes.

Answer:

```
traffic <- data.frame(
Road_ID=1:5,
Location=c("A","B","C","D","E"),
Vehicle_Count=c(120,98,145,180,160),
Average_Speed=c(45,38,52,41,47),
Signal_Status=c("Green","Red","Green","Yellow","Red"),
Timestamp=Sys.time()+1:5
)
```

Justification:

The selected attributes provide complete information about traffic conditions, allowing congestion analysis, speed monitoring and traffic signal optimization.

==========
MAKING DATA FRAMES - Q2
==========

Question:
Evaluate two alternative data frame designs for storing IoT sensor data and recommend the most scalable model.

Answer:

Design 1:
One column for each sensor.

Design 2:
Columns:
Sensor_ID
Timestamp
Sensor_Type
Reading

Recommended Model:

Sensor_ID | Timestamp | Sensor_Type | Reading

Justification:

The long-format design is more scalable because new sensors can be added without creating additional columns. It also supports filtering, aggregation and machine learning more efficiently.

==========
MAKING DATA FRAMES - Q3
==========

Question:
Analyze the consequences of assigning incorrect data types while creating a data frame for healthcare records.

Answer:

```
str(patient)

patient$Age <- as.numeric(patient$Age)
patient$BP <- as.numeric(patient$BP)
```

Justification:

Incorrect data types prevent statistical analysis, produce incorrect calculations and increase memory usage. Proper data types improve accuracy and computational efficiency.

==========
MAKING DATA FRAMES - Q4
==========

Question:
Create a strategy for constructing a data frame from multiple heterogeneous data sources while preserving consistency.

Answer:

```
csv_data <- read.csv("patients.csv")

excel_data <- read.csv("report.csv")

combined <- rbind(csv_data,excel_data)

str(combined)
```

Justification:

Standardize column names, data types and missing values before combining datasets to maintain consistency and avoid data loss.

==========
MAKING DATA FRAMES - Q5
==========

Question:
Assess the efficiency of creating a data frame from large vectors versus importing directly from files.

Answer:

```
df <- data.frame(
ID=1:100000,
Value=rnorm(100000)
)

file_df <- read.csv("largefile.csv")
```

Justification:

Creating data frames from vectors is suitable for generated data, while importing directly from files is more efficient for real-world datasets because it reduces memory usage and preserves existing data structures.

==========
WORKING WITH DATA FRAMES - Q1
==========

Question:
Analyze a scenario where filtering operations on a large data frame are producing unexpected results. Determine the root cause.

Answer:

```
str(df)

summary(df)

unique(df$Status)

subset(df, Status == "Active")
```

Justification:

Unexpected filtering results are usually caused by incorrect data types, leading/trailing spaces, missing values or incorrect logical conditions. Inspecting the data frame helps identify the root cause before filtering.

==========
WORKING WITH DATA FRAMES - Q2
==========

Question:
Evaluate different methods for sorting a million-row data frame and justify the most efficient approach.

Answer:

```
sorted_df <- df[order(df$Marks), ]

head(sorted_df)
```

Justification:

The order() function is efficient because it sorts using optimized internal algorithms and works well with large datasets while preserving row relationships.

==========
WORKING WITH DATA FRAMES - Q3
==========

Question:
A data analyst performs multiple transformations on a data frame. Analyze how errors can propagate through the workflow.

Answer:

```
str(df)

summary(df)

df$Marks <- as.numeric(df$Marks)

df <- na.omit(df)
```

Justification:

An error in one transformation can affect all subsequent operations. Validating the data after every transformation prevents incorrect results from propagating through the workflow.

==========
WORKING WITH DATA FRAMES - Q4
==========

Question:
Compare indexing and logical subsetting techniques for extracting data and evaluate their performance trade-offs.

Answer:

Indexing:

```
df[1:10, ]
```

Logical Subsetting:

```
df[df$Marks > 80, ]
```

Justification:

Indexing is faster when row positions are known, while logical subsetting is more flexible because it extracts records based on conditions. Logical subsetting is preferred for analytical tasks.

==========
WORKING WITH DATA FRAMES - Q5
==========

Question:
Design a workflow to monitor and validate modifications made to a shared data frame by multiple users.

Answer:

```
original_df <- df

updated_df <- df

identical(original_df, updated_df)
```

summary(updated_df)

Justification:

Maintaining an original copy, validating changes after each modification and comparing versions ensures data integrity and prevents accidental data loss in collaborative environments.

==========

MELTING AND CASTING OF DATA - Q1

==========

Question:
Analyze a dataset where repeated measurements are stored in multiple columns. Determine how melting improves analysis.

Answer:

```
library(reshape2)

long_df <- melt(df,
id.vars="ID",
variable.name="Measurement",
value.name="Value")

head(long_df)
```

Justification:

Melting converts wide-format data into long format, making filtering, grouping, visualization and statistical analysis easier. Each row represents one observation.

==========

MELTING AND CASTING OF DATA - Q2

==========

Question:
Evaluate the impact of casting data into different summary structures for business reporting.

Answer:

```
library(reshape2)

report <- dcast(
long_df,
```

```
Department ~ Product,
value.var="Sales",
fun.aggregate=sum
)

print(report)
```

Justification:

Casting summarizes data into a readable format, improving reporting and decision-making. Different aggregation functions (sum, mean, count) provide different business insights.

==========
MELTING AND CASTING OF DATA - Q3
==========

Question:
Compare two casting approaches that produce different aggregation results and explain the reasons.

Answer:

```
mean_report <- dcast(
long_df,
Department ~ Product,
value.var="Sales",
fun.aggregate=mean
)

sum_report <- dcast(
long_df,
Department ~ Product,
value.var="Sales",
fun.aggregate=sum
)
```

Justification:

Using **mean** calculates the average value, while **sum** calculates the total value. The choice depends on the analysis objective, so different aggregation functions naturally produce different results.

==========
MELTING AND CASTING OF DATA - Q4

==========

Question:
Design a workflow using melting and casting to prepare data for predictive modeling.

Answer:

```
library(reshape2)

long_df <- melt(df,id.vars="ID")

clean_df <- na.omit(long_df)

model_df <- dcast(
clean_df,
ID ~ variable,
value.var="value",
fun.aggregate=mean
)
```

Justification:

Melting standardizes the dataset, missing values are removed, and casting reconstructs the required structure for machine learning algorithms while preserving data consistency.

==========
MELTING AND CASTING OF DATA - Q5
==========

Question:
Assess the challenges of applying melt and cast operations on large-scale industrial datasets.

Answer:

Challenges:
- High memory usage
- Increased execution time
- Duplicate records
- Missing values
- Large number of variables

Solution:

Use efficient aggregation functions, remove unnecessary columns before reshaping, clean missing values, and process data in batches when handling very large datasets.

Justification:

Proper preprocessing reduces computation time and memory consumption while improving the accuracy and reliability of reshaped datasets.

==========
MERGING DATA FRAMES - Q1
==========

Question:
Analyze the consequences of performing an incorrect join operation between two financial datasets.

Answer:

```
merged_df <- merge(
finance1,
finance2,
by="Account_ID"
)
```

Justification:

An incorrect join may produce duplicate records, missing transactions, incorrect balances and misleading financial reports. Choosing the correct join type ensures accurate data integration.

==========
MERGING DATA FRAMES - Q2
==========

Question:
Evaluate inner, outer, left and right joins for integrating customer and transaction data.

Answer:

Inner Join:

```
merge(customer, transaction,
by="Customer_ID")
```

Left Join:

```
merge(customer, transaction,
by="Customer_ID",
all.x=TRUE)
```

Right Join:

```
merge(customer, transaction,
by="Customer_ID",
all.y=TRUE)
```

Outer Join:

```
merge(customer, transaction,
by="Customer_ID",
all=TRUE)
```

Justification:

- Inner Join returns only matching records.
- Left Join keeps all customer records.
- Right Join keeps all transaction records.
- Outer Join preserves all records from both datasets.

The appropriate join depends on the business requirement.

==========
MERGING DATA FRAMES - Q3
==========

Question:
A merged data frame contains unexpected duplicate rows. Investigate the possible causes and propose solutions.

Answer:

```
duplicated(merged_df)

unique(merged_df)

merged_df <- unique(merged_df)
```

Justification:

Duplicate rows may occur due to duplicate keys, one-to-many relationships or repeated source records. Removing duplicates and validating key columns ensures data integrity.

==========
MERGING DATA FRAMES - Q4
==========

Question:
Design a strategy to merge data frames from multiple departments while ensuring data consistency.

Answer:

```
sales <- read.csv("sales.csv")

hr <- read.csv("hr.csv")

finance <- read.csv("finance.csv")

merged_df <- Reduce(function(x,y)
merge(x,y,
by="Employee_ID",
all=TRUE),
list(sales,hr,finance))

str(merged_df)
```

Justification:

Standardize column names, data types and key fields before merging. Perform an outer join to preserve all departmental records and validate missing values after merging.

==========
MERGING DATA FRAMES - Q5
==========

Question:
Compare key-based merging and index-based merging techniques for large engineering datasets.

Answer:

Key-Based Merge:

merge(df1,df2,
by="Sensor_ID")

Index-Based Merge:

cbind(df1,df2)

Justification:

Key-based merging is more reliable because it matches records using unique identifiers and maintains data integrity. Index-based merging simply combines rows based on their position and may produce incorrect results if row orders differ. Therefore, key-based merging is recommended for large engineering datasets.